



Question 1 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated review CI jobs take 18 seconds to initialize before Claude begins analyzing code. Profiling reveals the delay comes from auto-discovery of hooks, MCP servers, plugins, skills, and multiple nested CLAUDE.md files throughout your monorepo. You need to cut startup time while ensuring reviews still enforce your team's coding standards, which are documented in your root-level CLAUDE.md file.

What is the most effective approach?

- ☐ A. Replace the default prompt entirely using `--system-prompt-file ./CLAUDE.md`, which bypasses default prompt assembly and loads only your project rules. 
- ☐ B. Run with `--bare` mode and specify all review criteria directly in the `-p` prompt argument for each CI invocation, without referencing any external files. 
- ☒ C. Run with `--bare` mode and pass `--append-system-prompt-file ./CLAUDE.md` to explicitly load your project standards while skipping all auto-discovery. 
- ☐ D. Keep the default initialization and add `--exclude-dynamic-system-prompt-sections` to reduce per- 



Question 2 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

In addition to your CI pipeline, your organization has enabled Claude's managed Code Review (via the Claude GitHub App) on this repository, and reviews run automatically on each PR. Reviews average 18 findings per PR. Developer feedback reveals three categories of unwanted noise: (1) style and formatting issues already enforced by your linter in CI, (2) findings on auto-generated template code under `src/gen/`, and (3) rendering helper patterns that are intentional project conventions but get flagged because they resemble common anti-patterns. Only about 4 findings per PR are genuine logic bugs.

What is the most effective way to reduce this noise while preserving detection of real issues?

- ☐ A. Add custom review instructions to a GitHub Actions workflow file, using the action's prompt parameter to suppress lint-duplicate findings, ignore generated template code, and apply stricter evidence requirements for rendering-related issues. 
- ☐ B. Configure separate GitHub Action workflow files for each code area, one for generated code with findings suppressed, one for rendering code with custom review instructions, and a general workflow for everything else. 

- ☐ B. Configure separate GitHub Action workflow files for each code area, one for generated code with findings suppressed, one for rendering code with custom review instructions, and a general workflow for everything else. 
- ☒ C. Create a REVIEW.md at the repository root with skip rules for CI-enforced checks and generated files, and a verification requirement that rendering-related findings must cite a specific line demonstrating incorrect behavior. 
- ☐ D. Add detailed explanations to the project's CLAUDE.md describing which patterns are intentional, that linting is handled separately by CI, and that the src/gen/ directory contains auto-generated template code. 



Question 3 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated reviewer uses a single prompt covering security issues, API design, and business logic correctness. Your evaluation suite shows strong recall for API design findings (82%) but poor recall for business logic edge cases in quiz scoring (34%). When you add few-shot examples of logic bugs to the prompt, logic recall improves to 41% but API design recall drops to 68%.

How should you address this trade-off to improve detection across both categories?

- ☐ A. Provide the full repository as context instead of just the changed files and surrounding code, giving the model deeper visibility into business logic patterns. 
- ☐ B. Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off. 
- ☐ C. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero in score calculations and boundary conditions in grading thresholds. 

boundary conditions in grading thresholds.

- D. Split the review into separate focused prompts — one for security and API design, another for business logic — each with dedicated examples, then consolidate findings before posting.



Question 4 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated code review is missing genuine bugs in pull requests. Investigation reveals that your review prompt includes the instruction: "Only flag critical issues that would definitely cause production failures. Ignore minor concerns and anything you're uncertain about." Developers confirm that some missed bugs are real logic errors the model investigated but chose not to report. The team requires that review output remain structured (each finding tagged with metadata) and actionable.

Which prompt change both removes the cause of the suppressed findings and preserves structured, tagged output for downstream filtering?

- ☒ A. Instruct the model to report all findings with a confidence level and severity tag, deferring filtering to a downstream step. 
- ☐ B. Remove all severity-related instructions from the prompt and let the model use its default judgment about what to report. 
- ☐ C. Add a second review pass that re-reads the diff using the same prompt, looking for anything the first pass may have missed. 



Question 5 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline reviews every PR using a single API call with a static prompt containing the diff and full text of each changed file — unchanged files are not included. Reviews are posted asynchronously and don't block PR creation. Developers report that reviews consistently miss bugs involving cross-file interactions — for example, a PR renames a function's parameters but the review doesn't flag callers in unchanged files that still use the old argument order. Evaluation shows cross-file bugs account for 35% of production incidents from reviewed PRs.

What is the most effective change to your review design?

- ☐ A. Use static analysis to build a dependency graph of changed code, then expand the prompt to include all files within two dependency hops of any changed file. 
- ☐ B. Run parallel review passes per changed file with direct dependents included in each pass, then aggregate and deduplicate findings using a final summarization call. 
- ☒ C. Redesign the review as a turn-limited agentic task where the model can read files and search the codebase via tools, following references to verify 

summarization call.

- ☒ C. Redesign the review as a turn-limited agentic task where the model can read files and search the codebase via tools, following references to verify cross-file findings.
- ☐ D. Add chain-of-thought instructions asking the model to list all external references in the diff, then reason step-by-step about how each change might affect callers in other files.





Question 6 of 60



You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.



Mark for Comment



Flag for Review

During initial testing of the automated review pipeline, you notice that reviews on large PRs (50+ changed files) sometimes take over 20 minutes and cost \$8-12 per run due to extensive agentic loops — Claude reads files, runs analysis tools, and iterates many times. Your team needs each invocation to abort once it reaches a fixed iteration count and a fixed dollar amount, enforced by Claude Code itself rather than the surrounding job runner.

Which configuration change directly enforces both of those per-invocation caps?

- ☐ A. Set ``timeout-minutes: 5`` on the GitHub Actions job step and monitor per-run costs via the Anthropic Console usage dashboard. 
- ☒ B. Add ``--max-turns 10 --max-budget-usd 2.00`` to the ``claude -p`` invocation to cap iterations and spend. 
- ☐ C. Set ``--permission-mode dontAsk`` to auto-deny any tool permission requests not in the explicitly allowed set. 
- ☐ D. Switch the ``--model`` flag to a smaller, cheaper model so each iteration uses fewer tokens and 



Question 7 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your test generation produces unit tests for new code, but reviews show 55% are low-value: trivial assertions that only verify functions don't throw exceptions, tests duplicating existing coverage, or tests ignoring your team's fixture conventions.

How do you reduce the rate of low-value tests being generated in the first place?

- ☐ A. Restrict test generation to directories where historical quality metrics show higher acceptance rates, disabling it for areas where generated tests consistently require heavy editing. 
- ☒ B. Document testing standards in CLAUDE.md including valuable test criteria, available fixtures with intended use cases, and examples distinguishing meaningful behavioral tests from trivial assertions. 
- ☐ C. Implement a two-phase generation where a second Claude call scores each test against quality criteria, filtering out low-scoring tests before presenting results to developers. 



Question 8 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline runs: ```` PROMPT="You are a code reviewer." PROMPT="$PROMPT Analyze the provided diff" PROMPT="$PROMPT for bugs, security issues," PROMPT="$PROMPT and style violations." claude -p \ -- dangerously-skip-permissions \ --system-prompt "$PROMPT" \ < diff.txt ```` The reviews complete and return feedback, but Claude only comments on the piped diff text — it never reads surrounding files in the checked-out repository to understand broader context, even when the diff modifies a function called by many other modules.

Which change to the invocation will cause Claude to read related files in the repository while still applying your custom review instructions?

- ☒ A. Keep `--system-prompt` and add `--allowedTools "Read,Glob,Grep"` so that the non-interactive `-p` mode permits file system tools that it otherwise disables. 
- ☐ B. Remove `--system-prompt` entirely and place the review instructions in a `CLAUDE.md` file at the repo root, since `--system-prompt` is incompatible with tool use under `-p`. 



Question 9 of 60



You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.



Mark for Comment



Flag for Review

The automated review consistently flags patterns your team uses intentionally — force-unwrapping optionals in test files, using large coordinator classes that follow your established architecture, and importing internally-maintained modules marked as deprecated in the public SDK. Developers are dismissing roughly 30% of all findings as project-specific false positives.

Which approach prevents the model from generating these findings in the first place by supplying the project's conventions as persistent context on every review?

- ☐ A. Configure the review to analyze only the changed lines in the diff without surrounding file context, reducing the amount of code the model evaluates per review. 
- ☐ B. Have developers add inline suppress-comments at flagged lines and preprocess diffs to exclude suppressed lines before sending code to the model. 
- ☐ C. Build post-processing keyword filters that suppress findings containing terms like "force unwrap," "large class," or "deprecated import" before results reach developers. 

- ☐ C. Build post-processing keyword filters that suppress findings containing terms like "force unwrap," "large class," or "deprecated import" before results reach developers. 
- ☒ D. Document the team's accepted patterns and intentional conventions in the project's CLAUDE.md file so the model receives this context during every review. 



Question 10 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated review calls the Claude API for each PR, using `tool_use` with a `report_findings` tool that returns a JSON array of finding objects (each with `file_path`, `line_number`, `severity`, `category`, and `description`). During testing on a large PR touching 30+ files, the response hits the `max_tokens` limit and the output is truncated mid-JSON, causing your pipeline's parser to fail.

What is the most effective way to handle this?

- ☐ A. Add retry logic that detects truncated JSON and re-sends the request with instructions to report only critical and high severity findings. 
- ☒ B. Split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays. 
- ☐ C. Increase `max_tokens` to the model's maximum and instruct Claude to keep finding descriptions under 50 words each. 
- ☐ D. Switch from `tool_use` to prompting Claude to return findings as a markdown list. 



Question 11 of 60



You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.



Mark for Comment



Flag for Review

Your CI pipeline performs security-focused code reviews on approximately 50 PRs daily, currently costing \$150/day using the synchronous API. Reviews are non-blocking—developers merge after tests pass and address findings in follow-up commits. You're evaluating the Message Batches API for its 50% cost reduction.

What factor most determines whether batch processing is appropriate for this use case?

- ☒ A. Whether review feedback arriving up to 24 hours after PR creation remains actionable. 
- ☐ B. Whether your result processing can handle reviews arriving in a different order than submitted. 
- ☐ C. Whether you can structure each review as a single request without multi-turn refinement. 
- ☐ D. Whether reducing per-review latency from 30-60 seconds to near-instant matters for your workflow. 



Question 12 of 60



You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.



Mark for Comment



Flag for Review

A developer uses Claude Code to refactor a function during their development session. Before committing, they ask the same Claude session to review the code for issues. Later, a separate automated CI review catches several bugs that the same-session review missed.

What best explains this discrepancy?

- ☐ A. The CI environment has access to the full codebase context while the local session only sees the current file 
- ☐ B. The CI review uses a more specific prompt tailored for catching bugs, while the developer's request was too general 
- ☒ C. Claude retains context about its prior reasoning in the session, making it less likely to question its own decisions 
- ☐ D. The extended session length caused the context window to fill with conversation history, leaving less room for thorough analysis 



Question 13 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated review generates many findings per PR, but developer feedback shows roughly half are dismissed as "not worth addressing." Analysis reveals dismissed findings are often technically accurate but involve minor style preferences or patterns acceptable in your codebase.

Before adding infrastructure complexity, what prompt design change would most effectively reduce dismissals while maintaining detection of genuine issues?

- ☒ A. Add explicit criteria defining which issues to report (bugs, security) versus skip (minor style, local patterns) 
- ☐ B. Add instructions asking Claude to rate each finding's confidence (1-10) and only include findings rated 8 or higher 
- ☐ C. Implement a secondary classification model that filters Claude's findings based on predicted developer acceptance 
- ☐ D. Append instructions to "only report findings you are highly confident are genuine problems" 



Question 14 of 60



Mark for Comment



Flag for Review



You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline includes a release notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API call using a Sonnet-tier Claude model. The release notes aren't needed until the following morning (results have ~12 hours of acceptable latency). Your team needs to reduce per-token API cost for this step while keeping the same model and prompts (no change to model tier or output quality).

Which approach satisfies all of these constraints?

- ☐ A. Issue the 200 Messages API requests in parallel using concurrent connections, since concurrency lowers the per-token price charged by the API. 
- ☒ B. Submit the 200 requests to the Message Batches API with unique custom_ids and retrieve results once the batch ends, which applies a 50% discount to all input and output tokens. 
- ☐ C. Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token rates. 



Question 15 of 60



Mark for Comment



Flag for Review

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

After deploying automated code review, developers report that approximately 35% of flagged findings are false positives falling into consistent patterns: style suggestions contradicting team conventions, security warnings for patterns safe in your deployment context, and performance suggestions that would degrade your specific use case. You want to reduce false positives while maintaining the ability to catch genuine issues.

Which approach best enables the model to generalize its judgment to novel code patterns it hasn't seen before?

- ☐ A. Create a comprehensive written specification of all patterns that should not be flagged, then include this full documentation in the system prompt. 
- ☐ B. Implement post-processing that uses keyword matching to filter out findings containing terms like "convention," "context-dependent," or "trade-off." 
- ☒ C. Include few-shot examples in your prompt showing annotated code snippets that distinguish acceptable patterns from genuine issues in each category. 
- ☐ D. Add instructions to your system prompt to "be conservative" and "only flag findings if you are 100% certain." 



Question 16 of 60



You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.



Mark for Comment



Flag for Review

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications.

What's the most effective approach to optimize for varying query complexity?

- ☒ A. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements. 
- ☐ B. Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination. 
- ☐ C. Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness. 



Question 17 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Production monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent—neither requires the other's output.

How should you modify the system to run these subagents concurrently?

- ☐ A. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator-subagent pair, then aggregates results. 
- ☒ B. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns. 
- ☐ C. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same time. 



Question 18 of 60



You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.



Mark for Comment



Flag for Review

Production monitoring shows that follow-up queries like "summarize what we learned about market trends" consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research.

What's the most effective way to improve response time for these follow-up summaries?

- ☐ A. Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings. 
- ☐ B. Pre-generate and cache summaries at multiple granularities whenever new findings accumulate. 
- ☒ C. Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analysis. 
- ☐ D. Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on demand. 



Question 19 of 60



You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.



Mark for Comment



Flag for Review

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow.

What change would most improve briefing quality?

- ☐ A. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation 
- ☐ B. Standardize all subagent outputs to JSON with fields for `claim`, `evidence`, `source`, and `confidence` 
- ☐ C. Standardize all subagent outputs to prose summaries with inline citations 
- ☒ D. Update the synthesis agent to render each content type appropriately: financial data as tables, news 

fields for 'claim', 'evidence', 'source', and 'confidence'

- ☐ C. Standardize all subagent outputs to prose summaries with inline citations
- ☒ D. Update the synthesis agent to render each content type appropriately—financial data as tables, news as prose



Question 20 of 60



You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.



Mark for Comment



Flag for Review

The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources.

What's the most effective way to improve subagent adaptability?

- ☐ A. Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure." 
- ☐ B. Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category. 
- ☒ C. Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy. 



Question 21 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The document analysis agent has a single `analyze_document` tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions.

What's the most effective way to improve reliability?

- ☐ A. Keep the single tool but add an `analysis_type` enum parameter requiring explicit selection between extraction, summarization, and verification modes 
- ☒ B. Split the generic tool into purpose-specific tools—`extract_data_points`, `summarize_content`, `verify_claim_against_source`—each with defined input/output contracts 
- ☐ C. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats 
- ☐ D. Have the coordinator pre-classify each analysis 



Question 22 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage.

What change would most effectively improve research completeness?

- ☒ A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again. 
- ☐ B. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information. 
- ☐ C. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator. 
- ☐ D. Have the report generation agent note which research questions couldn't be answered, so users 



Question 23 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate—it generates messages like "I'll ask the web search agent to find sources on this topic"—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors.

What is the most likely cause?

- ☐ A. The coordinator's `max_tokens` setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified. 
- ☐ B. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in `ClaudeAgentOptions`. 
- ☐ C. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked. 

list the available subagent types, preventing the model from knowing they can be invoked.

- D. The coordinator's `allowedTools` configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.





Question 24 of 60



You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.



Mark for Comment



Flag for Review

You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization—the synthesis agent attempts web searches, and the report generator tries to analyze documents.

What is the primary cause of this poor tool selection behavior?

- ☐ A. The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks. 
- ☐ B. The tool definitions consume too much context window space, leaving insufficient room for task content. 
- ☒ C. Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds. 
- ☐ D. The agents' role descriptions in their system prompts conflict with having access to tools outside that role. 



Question 25 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations.

What context-passing strategy provides the best balance of completeness and efficiency?

- ☐ A. Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated. 
- ☒ B. Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts. 
- ☐ C. Pass the full accumulated context from all prior agents. 
- ☐ D. Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only. 



Question 26 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations—the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps.

What's the most effective approach to ensure proper source attribution in the final reports?

- ☐ A. Have the report generator query the web search agent to re-locate sources for claims in the final report. 
- ☐ B. Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator. 
- ☐ C. Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format. 
- ☒ D. Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers). 

report generator.

- ☐ C. Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format. 
- ☒ D. Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers). 



Question 27 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely.

What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- ☐ A. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents 
- ☐ B. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity 
- ☒ C. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis 
- ☐ D. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations 



Question 28 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings.

What is the correct approach for providing the synthesis subagent with the information it needs?

- ☐ A. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks 
- ☐ B. Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results 
- ☐ C. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator 
- ☒ D. Include the complete findings from both subagents directly in the synthesis subagent's prompt 



Question 29 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings.

What state management approach best balances information fidelity with context efficiency when restoring agent state?

- ☒ A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts. 
- ☐ B. Have each agent maintain its own persistent state file and reload it independently at the start of each session. 
- ☐ C. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming. 
- ☐ D. Index all agent outputs in a shared vector store. When resuming, each agent queries the store using 



Question 29 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings.

What state management approach best balances information fidelity with context efficiency when restoring agent state?

- ☒ A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts. 
- ☐ B. Have each agent maintain its own persistent state file and reload it independently at the start of each session. 
- ☐ C. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming. 
- ☐ D. Index all agent outputs in a shared vector store. When resuming, each agent queries the store using
retrieval-augmented generation to reconstruct relevant prior findings. 



Question 30 of 60



Mark for Comment



Flag for Review

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), while other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate \$50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates \$35B ($\pm$ \$7B, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the market may be \$35B-\$50B depending on factors."

What systematic approach best addresses this?

- ☒ A. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological context. 
- ☐ B. Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator. 
- ☐ C. Add a verification subagent that cross-references findings across sources, only passing claims to 

☐ Mark for Comment ☐ Flag for Review



What exploration approach will most efficiently find the responsible code?

- ☐ A. Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service directory.
- ☐ B. Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.
- ☒ C. Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.
- ☐ D. Use Glob to find files in directories commonly associated with error handling (such as "errors/", "exceptions/", or "handlers/"), across all services.



Question 32 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Engineers frequently ask the agent to cross-reference code changes with Jira tickets during reviews—checking ticket descriptions, acceptance criteria, and recent comments. This currently requires manually copy-pasting content into conversations. The team wants the agent to access this standard Jira ticket data directly.

What's the most effective approach?

- ☒ A. Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces. 
- ☐ B. Build a custom MCP server wrapping Jira's API with tools designed specifically for this team's code review workflow. 
- ☐ C. Use the Bash tool with `curl` to call Jira's REST API, including authentication headers and parsing JSON responses inline. 
- ☐ D. Export Jira tickets to markdown files in the repository that the agent accesses using the Read tool. 



Question 33 of 60



Mark for Comment



Flag for Review



You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR.

Which task decomposition pattern is most appropriate for this workflow?

- ☐ A. Routing—classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category. 
- ☐ B. Orchestrator-workers—have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask. 
- ☐ C. Single comprehensive prompt—include all instructions in one prompt and let the model handle all three aspects simultaneously. 

- ☐ C. Single comprehensive prompt—include all instructions in one prompt and let the model handle all three aspects simultaneously.
- ☒ D. Prompt chaining—break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.



Question 34 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services.

What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- ☒ A. Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally. 
- ☐ B. Read all files containing "auth", "login", "permission", or "token" in their content or filename. 
- ☐ C. Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview. 
- ☐ D. Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system. 



Question 35 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent has spent 25 minutes exploring a game engine's rendering subsystem—reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific `VulkanPipeline` and `FrameGraph` classes it discovered earlier.

What's the most effective approach?

- ☐ A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation. 
- ☒ B. Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context. 
- ☐ C. Use `/clear` to reset context completely, then start fresh with physics exploration using file paths from the project's `CLAUDE.md`. 
- ☐ D. Continue in the current context with more targeted requests referencing the specific classes it discovered. 



Question 36 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

After adding an MCP server with specialized code refactoring tools (`extract_function`, `rename_variable`, `inline_function`), you notice the agent still uses basic text manipulation via `Write` and `Bash` `sed` commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like "`extract_function`: Extracts a function from code."

What's the most effective way to improve adoption of the MCP refactoring tools?

- ☐ A. Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them. 
- ☐ B. Accept this as expected behavior since simpler tools like `sed` are more predictable than specialized refactoring tools. 
- ☒ C. Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs. 



Question 37 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its `old_string` parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns.

What's the most reliable way to complete this insertion?

- ☐ A. Use Bash to append the function definition to the end of the file using heredoc syntax 
- ☐ B. Use Edit's `replace_all` parameter to target a common pattern and embed the new function in the replacement text 
- ☐ C. Use Edit with an extremely long `old_string` capturing 30+ lines of context to guarantee uniqueness 
- ☒ D. Use Read to load the file, add the function at the appropriate location, then Write the updated file 



Question 38 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

You're building a security scanning workflow.

When engineers need to locate all occurrences of a dangerous function like `eval()` across a large codebase, which tool should your agent use for content search?

- ☒ A. Use Grep to search for the pattern `"eval("` across all files in the codebase. 
- ☐ B. Read the project's main entry file and follow import statements to trace where `eval` might be used. 
- ☐ C. Use Glob with a pattern like `/**/*.eval*` to find files, then Read each matching file. 
- ☐ D. Use Bash to run ``ls -R | grep eval`` to recursively list files containing `eval`. 



Question 39 of 60



You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.



Mark for Comment



Flag for Review

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module).

What exploration strategy will most reliably identify all callers?

- ☐ A. Use Grep to search for the function's original name across the codebase. 
- ☐ B. Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points. 
- ☒ C. Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase. 
- ☐ D. Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function. 



Question 40 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration.

What's the most effective approach?

- ☐ A. Launch a fresh subagent with a summary of prior findings. 
- ☐ B. Resume the subagent from its previous transcript without mentioning the changes—the architecture understanding remains valid. 
- ☒ C. Resume the subagent from its previous transcript and inform it about the renamed functions. 
- ☐ D. Launch a fresh subagent and include the prior transcript in the initial prompt for context. 



Question 41 of 60



You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.



Mark for Comment



Flag for Review

Your productivity agent connects to three MCP servers: an issue tracker (search_issues, get_issue, create_comment), a documentation wiki (search_docs, get_page, list_spaces), and a database explorer (run_query, get_schema, list_databases). When engineers ask cross-system questions like "What database tables are affected by the authentication refactor in PROJ-1234?", monitoring shows the agent makes 8-10 sequential tool calls, frequently issues exploratory calls because it lacks visibility into what content each server contains, and exhausts context space before completing complex investigations.

What architectural change best leverages MCP capabilities to address these issues?

- ☐ A. Consolidate all three servers into a unified MCP server with cross-referencing capabilities 
- ☒ B. Expose each server's content catalog as MCP resources—issue summaries, documentation hierarchy, database schemas 
- ☐ C. Add a prepare_investigation tool to each server that accepts a natural language question and returns relevant content summaries 



Question 42 of 60



Mark for Comment



Flag for Review



You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search.

When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

- ☐ A. The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes. 
- ☐ B. You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time. 
- ☒ C. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent. 
- ☐ D. The agent automatically selects the most relevant server based on the request and loads only that server's tools. 



Question 43 of 60



You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.



Mark for Comment



Flag for Review

An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total).

What's the most effective next step for building understanding while managing context constraints?

- ☒ A. Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations. 
- ☐ B. Use Glob to find files matching common caching patterns (*cache*.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps. 
- ☐ C. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation. 
- ☐ D. Use Grep to search for "invalidate" and "expire" patterns across all files, then Read each file. 



Question 44 of 60



Mark for Comment



Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about code structure it discussed earlier. Engineers report having to repeat context about modules they've already explored.

What's the most effective approach to address this?

- ☒ A. Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions. 
- ☐ B. Create summaries of all source files before exploration begins, loading only these compressed representations into context. 
- ☐ C. Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data. 
- ☐ D. Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context. 



Question 45 of 60



You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.



Mark for Comment



Flag for Review

An engineer asks the agent to find all files in the monorepo that import the `@company/auth` package to understand how authentication is used across services.

Which built-in tool is most appropriate for this task?

- ☒ A. Grep, to search for the import statement pattern across file contents 
- ☐ B. Glob, to find files with "auth" in their filename or path 
- ☐ C. Bash, to execute `find . -type d -name "*auth*"` and explore matching directories 
- ☐ D. Read, starting with package.json files to trace dependency declarations 



Question 46 of 60



You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.



Mark for Comment



Flag for Review

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article—for example, outputting "500" for attendee_count when the source contains no attendance information.

What's the most effective way to reduce these false extractions?

- ☒ A. Add prompt instructions to return null for any field where information is not directly stated in the source. 
- ☐ B. Add a post-processing step using a second LLM call to verify each extracted value exists in the source document. 
- ☐ C. Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data. 
- ☐ D. Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies. 



Question 47 of 60



You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.



Mark for Comment



Flag for Review

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by custom_id.

What's the most cost-effective approach to process these failures?

- ☐ A. Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts 
- ☐ B. Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch 
- ☒ C. Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions 
- ☐ D. Resubmit the entire 10,000 document batch using a model tier with a larger context window 



Question 48 of 60



You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.



Mark for Comment



Flag for Review

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements.

How should you architect the processing pipeline?

- ☐ A. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return. 
- ☒ B. Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API. 
- ☐ C. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types. 
- ☐ D. Submit all documents to the Batch API with custom_ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions. 



Question 49 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions ($\geq 85\%$) also contain errors—cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time.

What approach is most effective?

- ☒ A. Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection. 
- ☐ B. Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score. 
- ☐ C. Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review. 



Question 50 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the "materials" field—sometimes returning "cotton blend", other times "Cotton/Polyester mix", and occasionally omitting the field when material information is clearly present in the source.

What's the most effective way to improve extraction consistency?

- ☐ A. Set temperature to 0 to eliminate randomness and ensure deterministic outputs 
- ☐ B. Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability 
- ☐ C. Make the "materials" field required instead of optional in the schema to force the model to always extract a value 
- ☒ D. Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats 



Question 51 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system uses tool_use with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model's context window is 200K tokens.

What is the most likely cause?

- ☐ A. Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions. 
- ☐ B. The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus. 
- ☐ C. Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length. 



Question 52 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide.

What's the most effective way to ensure structured metadata extraction happens first?

- ☐ A. Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur. 
- ☐ B. Set `tool_choice` to `"auto"` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools. 
- ☐ C. Set `tool_choice` to `"any"` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`. 

- ☐ C. Set `tool_choice` to `"any"` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`. 
- ☒ D. Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata. 



Question 53 of 60



You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.



Mark for Comment



Flag for Review

Your extraction uses tool use with a JSON schema where ``property_type`` is defined as an enum: ``['house', 'apartment', 'condo', 'townhouse']``. After deployment, 8% of extractions fail schema validation. Investigation reveals listings mention many uncommon property types—"studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse"—and new types continue appearing regularly.

What's the most effective long-term solution?

- ☐ A. Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases. 
- ☐ B. Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value. 
- ☐ C. Change ``property_type`` from an enum to a free-form string and implement a normalization step in post-processing. 
- ☒ D. Add an "other" value to your enum with a separate ``property_type_detail`` string field for specifics when ``property_type`` is "other". 

types to the closest existing enum value.

- ☐ C. Change ``property_type`` from an enum to a free-form string and implement a normalization step in post-processing. 
- ☒ D. Add an "other" value to your enum with a separate ``property_type_detail`` string field for specifics when "other" is selected. 



Question 1 of 60



You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.



Mark for Comment



Flag for Review

Your automated review CI jobs take 18 seconds to initialize before Claude begins analyzing code. Profiling reveals the delay comes from auto-discovery of hooks, MCP servers, plugins, skills, and multiple nested CLAUDE.md files throughout your monorepo. You need to cut startup time while ensuring reviews still enforce your team's coding standards, which are documented in your root-level CLAUDE.md file.

What is the most effective approach?

- ☐ A. Replace the default prompt entirely using `--system-prompt-file ./CLAUDE.md`, which bypasses default prompt assembly and loads only your project rules. 
- ☐ B. Run with `--bare` mode and specify all review criteria directly in the `-p` prompt argument for each CI invocation, without referencing any external files. 
- ☒ C. Run with `--bare` mode and pass `--append-system-prompt-file ./CLAUDE.md` to explicitly load your project standards while skipping all auto-discovery. 
- ☐ D. Keep the default initialization and add `--exclude-dynamic-system-prompt-sections` to reduce per- 



Question 54 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence $\geq 90\%$ have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions.

Before deploying, what validation step is most critical?

- ☐ A. Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports. 
- ☒ B. Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate. 
- ☐ C. Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors. 
- ☐ D. Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data. 



Question 55 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON.

What approach provides the most reliable schema compliance?

- ☐ A. Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON. 
- ☐ B. Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails. 
- ☒ C. Define a tool with your target schema as input parameters and have Claude call it with the extracted data. 
- ☐ D. Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response. 



Question 56 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After deployment, you find that 12% of extractions contain semantic errors that pass JSON schema validation (e.g., a duration like "30 minutes" incorrectly placed in an ingredient quantity field). Human reviewers have capacity to check only 20% of extractions.

Which approach most effectively allocates reviewer attention?

- ☒ A. Have the model output field-level confidence scores, then calibrate review thresholds using a labeled validation set. 
- ☐ B. Prioritize review of all extractions where required fields are empty or explicitly marked as not found. 
- ☐ C. Review all extractions from documents with formatting anomalies such as unusual layouts or mixed content types. 
- ☐ D. Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns. 



Question 57 of 60



You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.



Mark for Comment



Flag for Review

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned."

What's the most effective improvement?

- ☐ A. Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata. 
- ☐ B. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries. 
- ☐ C. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills." 
- ☒ D. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity. 



Question 58 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline occasionally receives responses that don't parse as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it.

What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

- ☐ A. Implement a retry loop that catches JSON parse errors and re-prompts Claude with the error details, asking it to correct the malformed output. 
- ☒ B. Define a tool with a JSON schema specifying the expected structure, using `tool_use` to constrain Claude's output to schema-compliant JSON. 
- ☐ C. Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text. 
- ☐ D. Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the 



Question 59 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats—inline citations vs. bibliographies, methodology sections vs. details embedded in introductions.

What's the most effective way to address these failures?

- ☐ A. Implement retry logic that re-sends requests when validation detects empty required fields. 
- ☐ B. Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract. 
- ☐ C. Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation. 

When the model fails to extract.

- ☐ C. Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.
- ☒ D. Add few-shot examples demonstrating extractions from documents with varied structures—showing how to identify citations in different formats and locate methodology details across section types.





Question 60 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your invoice extraction uses tool use with strict JSON schemas. JSON syntax errors never occur, but 12% of extractions fail semantic validation—for example, line item amounts don't sum to the extracted total, or vendor IDs don't match valid formats. These failures currently route to manual review.

What's the most effective approach to reduce manual review volume while maintaining accuracy?

- ☒ A. When validation fails, make a follow-up request with the document, extraction, and validation errors for model correction. 
- ☐ B. Add stricter schema constraints with detailed field descriptions to prevent the model from generating invalid values initially. 
- ☐ C. Retry the extraction up to 3 times when validation fails, accepting the first result that passes validation. 
- ☐ D. Implement post-processing logic that automatically corrects common errors, such as recalculating totals from line items when sums don't match. 

EXAM: Claude Certified Architect Foundations

This report shows the percentage of items in each section you answered correctly for the Claude Certified Architect Foundations

PASSING SCORE: 720

YOUR SCORE: 988

GRADE: Pass

TEST OBJECTIVES

PERCENT CORRECT

Configure agentic loops to emit multiple tool calls within a single response turn, enabling parallel execution of independent subtasks and reducing total round-trip latency.

100%

Apply session resumption techniques—including targeted re-analysis of changed files and context injection—to restore agent state accurately without repeating prior work.

100%

Construct subagent prompts that include all findings, structured data, and source metadata required for task completion without returning to the coordinator for missing context.

100%

Evaluate subagent delegation strategies—goal-oriented versus procedural instructions—and select the approach that enables adaptive behavior while maintaining coordinator visibility and control.

100%

Configure subagent invocations with appropriate tool restrictions, context scoping, and system prompts that constrain each agent to its designated role.

100%